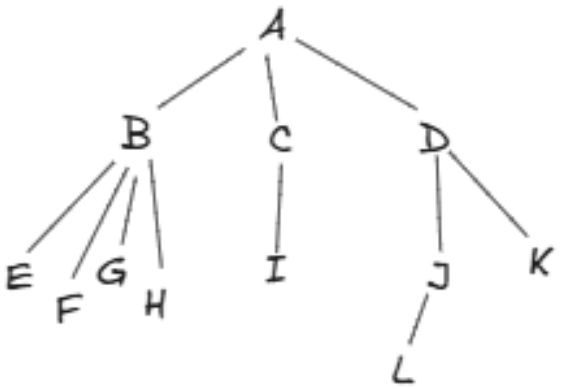
Ejercicios Arboles

Recorridos de árboles

Ejemplo:



Criterios de resolución (según la guía)

- Declarar siempre variables en bloque `Var` con formato `Tipo: variable`.
- Usar asignación con `<-` y comparaciones con `=`, `!=`, `>=`, `<=`.
- En árboles n-arios usar representación hijo-izquierdo/hermano-derecho.
- Si el enunciado exige punteros: recorrer con `pointer to Nodo...` y `->`.
- Si el enunciado prohíbe punteros: resolver con TADs (`ArbolBin`, `ArbolN`, `Lista`, `Cola`, `Pila`) y getters.

PreOrden:

1. Procesar Raíz
2. Procesar hijos en PreOrden (y hermanos)

```
func ArbolN<Element>::preOrden(): Lista<Element>
   Var
       Lista<Element>: result
       pointer to NodoN<Element>: ptr
   Begin
       result.construir()
       ptr <- instance.nodoRaiz
       instance.preOrden(ptr, result)
       return result
endfunc

func ArbolN<Element>::preOrden(pointer to NodoN<Element>: ptr, ref Lista<Element>:
result)
   Var
       pointer to NodoN<Element>: hijo
   Begin
       if ptr then
           result.insertar(ptr->getInfo(), result.getLong() + 1)
           hijo <- ptr->getHijoIzq()
           while hijo do
               instance.preOrden(hijo, result)
               hijo <- hijo->getHnoDer()
           endwhile
       endif
endfunc
```

PostOrden:

1. Procesar hijos en PostOrden (y hermanos)
2. Procesar raíz

```
func ArbolN<Element>::postOrden(): Lista<Element>
   Var
       Lista<Element>: result
       pointer to NodoN<Element>: ptr
   Begin
       result.construir()
       ptr <- instance.nodoRaiz
       instance.postOrden(ptr, result)
       return result
endfunc

func ArbolN<Element>::postOrden(pointer to NodoN<Element>: ptr, ref Lista<Element>:
result)
   Var
       pointer to NodoN<Element>: hijo
   Begin
       if ptr then
           hijo <- ptr->getHijoIzq()
           while hijo do
               instance.postOrden(hijo, result)
               hijo <- hijo->getHnoDer()
           endwhile
           result.insertar(ptr->getInfo(), result.getLong() + 1)
       endif
endfunc
```

InOrden:

1. Procesar primer hijo
2. Procesar raíz
3. Procesar resto de hijos en InOrden

```
func ArbolN<Element>::inOrden(): Lista<Element>
   Var
       Lista<Element>: result
       pointer to NodoN<Element>: ptr
   Begin
       result.construir()
       ptr <- instance.nodoRaiz
       instance.inOrden(ptr, result)
       return result
endfunc

proc ArbolN<Element>::inOrden(pointer to NodoN<Element>: ptr, ref Lista<Element>:
result)
   Var
       pointer to NodoN<Element>: hijo
   Begin
       if ptr then
           hijo <- ptr->getHijoIzq()
           if hijo then
               instance.inOrden(hijo, result)
               hijo <- hijo->getHnoDer()
           endif

           result.insertar(ptr->getInfo(), result.getLong() + 1)

           while hijo do
               instance.inOrden(hijo, result)
               hijo <- hijo->getHnoDer()
           endwhile
       endif
endproc
```

Por Niveles:

Recorrido de árbol por niveles

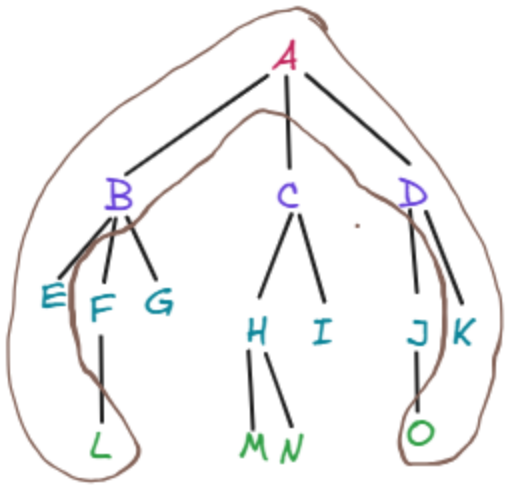
```
func ArbolN<Element>::porNiveles(): Lista<Element>
   Var
       Cola<pointer to NodoN<Element>>: colaAux
       Lista<Element>: result
       pointer to NodoN<Element>: ptr, hijo
   Begin
       result.construir()
       colaAux.construir()
       if ¬instance.nodoRaiz then
           return result
       endif

       colaAux.encolar(instance.nodoRaiz)
       while ¬colaAux.esVacia() do
           ptr <- colaAux.getFrente()
           colaAux.desencolar()
           result.insertar(ptr->getInfo(), result.getLong() + 1)

           hijo <- ptr->getHijoIzq()
           while hijo do
               colaAux.encolar(hijo)
               hijo <- hijo->getHnoDer()
           endwhile
       endwhile
       return result
endfunc
```

Ejercicios Arboles

Parcial 1-2024

Ejercicio 1: Suma Interna

Se define la “frontera” de un arbol n-ario como aquellos nodos en la parte externa del árbol (ver
sombreado):



Se requiere que implemente la función:
`func ArbolN<entero>::sumaInterna(): entero`
Que sume todos los nodos internos, o sea, aquellos que NO pertenecen a la frontera.

> La solución debe ser a lo sumo O(n). UTILICE APUNTADORES EN SU SOLUCION.

```
func ArbolN<int>::sumaInterna(): int
   Var
       Lista<Lista< pointer to NodoN<int> >>: switch
       Lista<pointer to NodoN<int> >: listaAux
       Lista<int>: elements
       int: result, recorrido, llenado
       pointer to NodoN<int>: ptr, hijo
   Begin
       elements.construir()
       listaAux.construir()
       switch.construir()
       switch.insertar(listaAux, 1)
       swtich.insertar(listaAux, 1)
       switch.consultar(1).insertar(instance.nodoRaiz,1)
       result <- 0
       recorrido <- 1
       llenado <- 2
       ptr <- instance.nodoRaiz

       while ¬switch.consultar(recorrido).esVacia() do
           while ¬switch.consultar(recorrido).esVacia() do
               // Procesar Lista/Cola
               ptr <- switch.consultar(recorrido).consultar(1)
               switch.consultar(1).eliminar(1)

               // Procesar hijos y hermanos
               hijo <- ptr.getHijoIzq()
               while hijo do
                   // Inserta en la otra lista para cambiar entre las dos
                   switch.consultar(llenado).insertar(hijo,
switch.consultar(llenado).getLong() +1)
                   elements.insertar(hijo.getInfo(), 1) // Como se debe eliminar la
frontera, no impora el orden.
                   hijo <- hijo.getHnoDer()
               endwhile
           endwhile

           // Eliminar la frontera y calcular la suma
           if ¬elements.esVacia() then
               elements.eliminar(1)
           endif
           if ¬elements.esVacia() then
               elements.eliminar(elements.getLong() +1)
           endif
           while ¬elements.esVacia() then
               result <- result + elements.consultar(1)
               elements.eliminar(1)
           endwhile
```

```
            // Invertir recorrido y llenado
            recorrido <- recorrido + llenado
            llenado <- recorrido - llenado
            recorrido <- recorrido - llenado
        endwhile
        result result
endfunc
```

> La inversión de recorrido y llenado se podría hacer de esta manera también:
>
> ```
> if recorrido = 1 then
>  recorrido <- 2
>  llenado <- 1
> else
>  recorrido <- 1
>  llenado <- 2
> endif
> ```

Ejercicio 2: camino entre dos nodos
Dado un Dado un árbol N-ario y dos elementos n y m pertenecientes al árbol. Encontrar el camino entre ellos dos. El prototipo de su función debe ser:
`func ArbolN<Element>::camino(Element: n, m): Lista<Element>`

> Su solución debe ser a lo sumo O(n). SOLO APUNTADORES EN SU SOLUCION.

¿Cómo funciona la idea?

1. Se construye la ruta desde la raíz hasta `n`.
2. Se construye la ruta desde la raíz hasta `m`.
3. Se comparan ambas rutas desde el inicio para encontrar el último nodo común (ancestro común más cercano).
4. Con lo que sobra de la ruta de `n` se arma el tramo de subida hacia el ancestro (insertando al principio para invertir).
5. Luego se agrega el ancestro y después el tramo restante de la ruta de `m`.

En la función auxiliar `buscarRuta`, la lista `ruta` siempre representa el camino actual desde la raíz hasta el nodo que se está visitando.

- Cuando se entra a un nodo, se inserta en `ruta`.
- Si ese nodo es el objetivo, se marca `found`.
- Si no se encuentra en sus hijos, se elimina de `ruta` para deshacer esa rama.

Con esto, cuando termina la búsqueda, la `ruta` queda exactamente con el camino raíz → objetivo.

Complejidad:

- `buscarRuta` visita cada nodo una vez por búsqueda: O(n).
- Se ejecuta dos veces (para `n` y para `m`) y luego se combinan rutas linealmente.
- Complejidad total: O(n).

```
func ArbolN<Element>::camino(Element: n, m): Lista<Element>
   Var
       Lista<pointer to NodoN<Element>>: rutaN, rutaM
       Lista<Element>: result
       bool: found
       pointer to NodoN<Element>: ancestro
   Begin
       rutaN.construir()
       rutaM.construir()
       result.construir()
       ancestro <- NULL

       found <- false
       instance.buscarRuta(instance.nodoRaiz, n, rutaN, found)

       found <- false
       instance.buscarRuta(instance.nodoRaiz, m, rutaM, found)

       if rutaN.esVacia() v rutaM.esVacia() then
           return result
       endif

       while ¬rutaN.esVacia() ^ ¬rutaM.e0sVacia() ^ rutaN.consultar(1) =
rutaM.consultar(1) do
           ancestro <- rutaN.consultar(1)
           rutaN.eliminar(1)
           rutaM.eliminar(1)
       endwhile

       while ¬rutaN.esVacia() do
           result.insertar(rutaN.consultar(1)->getInfo(), 1)
           rutaN.eliminar(1)
       endwhile

       if ancestro then
           result.insertar(ancestro->getInfo(), result.getLong() + 1)
       endif

       while ¬rutaM.esVacia() do
           result.insertar(rutaM.consultar(1)->getInfo(), result.getLong() + 1)
           rutaM.eliminar(1)
       endwhile

       return result
endfunc

proc ArbolN<Element>::buscarRuta(pointer to NodoN<Element>: ptr, Element: x, ref
Lista<pointer to NodoN<Element>>: ruta, ref bool: found)
```

```
    Var
        pointer to NodoN<Element>: hijo
    Begin
        if ptr ^ ¬found then
            ruta.insertar(ptr, ruta.getLong() + 1)

            if ptr->getInfo() = x then
                found <- true
            else
                hijo <- ptr->getHijoIzq()
                while hijo ^ ¬found do
                    instance.buscarRuta(hijo, x, ruta, found)
                    hijo <- hijo->getHnoDer()
                endwhile

                if ¬found then
                    ruta.eliminar(ruta.getLong())
                endif
            endif
        endif
endproc
```

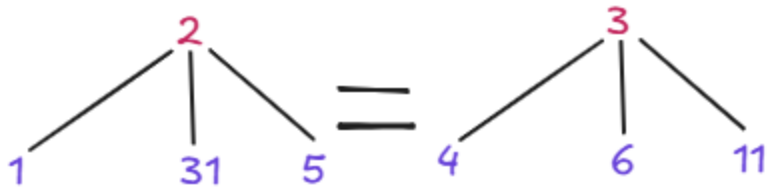
Parcial 2-2023

Ejercicio 1: Arboles Isomorfos

Se define el isomorfismo de arboles como una biyección de los nodos de un grafo sobre otro, de modo que se preserva la adyacencia de los vértices. Más formalmente, el isomorfismo de dos arboles A y B es una biyección f entre los conjuntos de sus nodos f: $N(A) \rightarrow N(B)$ que preserva la relación de adyacencia. Es decir, cualquier par de nodos u y v de A son adyacentes si y solo si lo son sus imágenes, $f(u)$ y $f(v)$ en B.

Esta definición matemática del problema es muy elegante, formal y puede causar confusión y

temor en el estudiante que está presentando esta evaluación. Sin embargo, solo fue colocada para ocupar espacio en la hoja. Realmente dos árboles son isomorfos si y sólo si su arquitectura es igual sin importar el contenido de sus nodos. Ejemplo:



Implemente el método:

`func ArbolBin<entero>::esIsomorfo(ArbolBin<entero>: b): logico`

que retorna verdadero si ambos árboles son isomorfos.

> UTILICE SOLO APUNTADORES en su solución y esta debe ser a lo sumo O(n).

```
func ArbolBin<int>::esIsomorfo(ArbolBin<int>: b): bool
   Begin
       return instance.esIsomorfo(instance.nodoRaiz, b.nodoRaiz)
endfunc

func ArbolBin<int>::esIsomorfo(pointer to NodoBin<int>: ptr1, ptr2): bool
   Begin
       if ¬ptr1 ^ ¬ptr2 then
           return true
       endif

       if ¬ptr1 v ¬ptr2 then
           return false
       endif

       return instance.esIsomorfo(ptr1->getHijoIzq(), ptr2->getHijoIzq()) ^
instance.esIsomorfo(ptr1->getHijoDer(), ptr2->getHijoDer())
endfunc
```

Ejercicio 2: Princesa más cercana

Suponemos la clase “Personaje” con dos métodos: “esDragon()” y “esPrincesa()” que permiten determinar si un personaje es un dragón o es una princesa, respectivamente. Suponemos también que ningún personaje es dragón y princesa a la vez, pero que un personaje puede no ser ninguna de las dos cosas. Dado un árbol n-ario A cuyos elementos son del tipo “Personaje”, se denominan nodos accesibles aquellos nodos tales que a lo largo del camino desde la raíz hasta el nodo (ambos inclusive) no se encuentra ningún dragón.

Crear una función:

`func princesaMasCercana(ArbolN<Personaje>: arbol): string`

que encuentre el nombre de una princesa lo más cerca posible de la raíz de A.

> Suponiendo que algún nodo accesible de la raíz contiene una princesa. A lo sumo O(n). UTILICE APUNTADORES.

```
func ArbolN<Personaje>::princesaMasCercana(): string
   Var
       Cola<pointer to NodoN<Personaje>>: colaAux
       pointer to NodoN<Personaje>: hijo
   Begin
       colaAux.construir()
       colaAux.encolar(instance.nodoRaiz)
       while ¬colaAux.esVacia() do
           // Procesar nodo
           if colaAux.getFrente()->getInfo().esPrincesa() then
               return colaAux.getFrente()->getInfo().getNombre()
           endif
           // Insertar hijos en cola
           hijo <- colaAux.getFrente()->getHijoIzq()
           while hijo doh
               if ¬hijo->getInfo().esDragon() then
                   colaAux.encolar(hijo)
               endif
               hijo <- hijo->getHnoDer()
           endwhile

           colaAux.desencolar()
       endwhile
       return "" // si no lo encuentra retorna string vacio
endfunc
```

Parcial 2-2025 mod1

Ejercicio 1: nivel con suma maxima
Dado un árbol n-ario. Implemente el método:
`func ArbolN<int>::getMaxLevel(): Lista<int>`
Que devuelva la lista de elementos que conforman el nivel cuya suma de sus elementos sea máxima.

> La solución debe ser a lo sumo O(n) y DEBE USAR APUNTADORES.
> *Version de Doble buffer y lista auxiliar (misma logica de suma interna)*

```
func ArbolN<int>::getMaxLevel(): Lista<int>
    Var
        Lista<Lista<pointer to NodoN<int>>>: switch
        Lista<pointer to NodoN<int>>: listaAux
        Lista<int>: result
        pointer to NodoN<int>: ptr, hijo
        int: sum, max, recorrido, llenado

    Begin
        result.construir()
        listaAux.construir()
        swtich.construir()
        switch.insertar(listaAux, 1)
        switch.insertar(listaAux, 1)
        switch.consultar(1).insertar(instance.nodoRaiz, 1)
        ptr <- instance.nodoRaiz
        recorrido <- 1
        llenado <- 2
        sum <- 0
        max <- -inf

        while ¬switch.consultar(recorrido).esVacia() do
            sum <- 0
            while ¬switch.consultar(recorrido).esVacia() do
                // Procesar Raiz
                ptr <- switch.consultar(recorrido).consultar(1)
                switch.consultar(recorrido).eliminar(1)
                sum <- sum + ptr->getInfo()

                // Guardado temporal del resultado
                listaAux.insertar(ptr->getInfo(), listaAux.getLong() +1)

                // Procesar hijos
                hijo <- ptr->getHijoIzq()
                while hijo do
                    switch.consultar(llenado).insertar(hijo,
switch.consultar(llenado).getLong() +1)
                    hijo <- hijo->getHnoDer()
                endwhile
            endwhile

            // Procesamiento de datos
            if sum > max then
                max <- sum
                // Vaciado de lista temporal hacia result
                result.vaciar()
                while ¬listaAux.esVacia() do
                    result.insertar(listaAux.consultar(1), result.getLong() +1)
```

```
                    listaAux.eliminar(1)
                endwhile
            else
                listaAux.vaciar()
            endif

            // Inversion de colas
            recorrido <- recorrido + llenado
            llenado <- recorrido - llenado
            recorrido <- recorrido - llenado
        endwhile

        // Retorno de resultado
        return result
endfunc
```

Version de una sola cola y tamaño de nivel

```
func ArbolN<int>::getMaxLevel(): Lista<int>
    Var
        Cola<pointer to NodoN<int>>: colaAux
        Lista<int>: nivelAux, result
        pointer to NodoN<int>: ptr, hijo
        int: tamNivel, i, sumaNivel, max
    Begin
        colaAux.construir()
        nivelAux.construir()
        result.construir()

        colaAux.encolar(instance.nodoRaiz)
        max <- -inf

        while ¬colaAux.esVacia() do
            tamNivel <- colaAux.getLong()
            sumaNivel <- 0
            nivelAux.vaciar()

            for i <- 1 to tamNivel do
                ptr <- colaAux.getFrente()
                colaAux.desencolar()

                sumaNivel <- sumaNivel + ptr->getInfo()
                nivelAux.insertar(ptr->getInfo(), nivelAux.getLong() + 1)

                hijo <- ptr->getHijoIzq()
                while hijo do
                    colaAux.encolar(hijo)
                    hijo <- hijo->getHnoDer()
                endwhile
            endfor

            if sumaNivel > max then
                max <- sumaNivel
                result.vaciar()
                while ¬nivelAux.esVacia() do
                    result.insertar(nivelAux.consultar(1), result.getLong() + 1)
                    nivelAux.eliminar(1)
                endwhile
            endif
        endwhile

        return result
endfunc
```

Ejercicio 2: camino entre u y v
Dado un nodo x y dos nodos u y v, queremos determinar si el nodo x pertenece al único camino entre u y v.
`func ArbolBin<int>::estaEnCamino(x: int, u: int, v: int): bool`
que devuelva true si el nodo x está en el camino entre u y v, y false en caso contrario.

> La solución debe ser a lo sumo O(n) y DEBE USAR APUNTADORES.

La idea más limpia con LCA es:

1. Calcula w = LCA(u, v).
2. x está en el camino u-v si y solo si:
- x está debajo de w (w es ancestro de x), y
- x es ancestro de u o x es ancestro de v.

Fórmula:

- esta = esAncestro(w, x) ^ (esAncestro(x, u) v esAncestro(x, v))

Por qué funciona:

1. El camino u-v es unión de dos tramos:
   - w hasta u
   - w hasta v
2. Si x está en uno de esos tramos, cumple esa condición.
3. Si x está fuera de esos tramos (por ejemplo, en otra rama), falla.

Función auxiliar clave
esAncestro(a, b):

- true si a aparece en el camino desde raíz hasta b (equivalente: b está en subárbol de a).
- Recursiva:
  1. si a nulo → false
  2. si a.info = b → true
  3. retorna esAncestro(a.izq, b) v esAncestro(a.der, b)

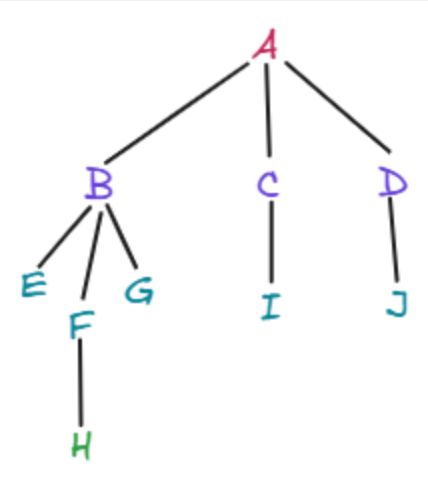
```
func ArbolBin<int>::estaEnCamino(int: x, u, v): bool
    Var
        pointer to NodoBin<int>: w, px
    Begin
        // w = LCA(u, v)
        w <- instance.lcaPtr(instance.nodoRaiz, u, v)
        if ¬w then
            return false
        endif

        // Buscar el nodo x
        px <- instance.buscarNodo(instance.nodoRaiz, x)
        if ¬px then
            return false
        endif

        // x debe estar bajo w, y ademas x debe ser ancestro de u o de v
        return instance.esAncestro(w, x) ^ (instance.esAncestro(px, u) v
instance.esAncestro(px, v))
endfunc

func ArbolBin<int>::buscarNodo(pointer to NodoBin<int>: ptr, int: objetivo): pointer
to NodoBin<int>
    Var
        pointer to NodoBin<int>: izq, der
    Begin
        if ¬ptr then
            return NULL
        endif

        if ptr->getInfo() = objetivo then
            return ptr
        endif

        izq <- instance.buscarNodo(ptr->getHijoIzq(), objetivo)
        if izq then
            return izq
        endif

        der <- instance.buscarNodo(ptr->getHijoDer(), objetivo)
        return der
endfunc

func ArbolBin<int>::esAncestro(pointer to NodoBin<int>: ptr, int: objetivo): bool
    Begin
        if ¬ptr then
            return false
        endif
```

```
        if ptr->getInfo() = objetivo then
            return true
        endif

        return instance.esAncestro(ptr->getHijoIzq(), objetivo) v
instance.esAncestro(ptr->getHijoDer(), objetivo)
endfunc
```

Ejercicios Varios

Obtener primos

Dado un árbol n-ario y un nodo `x`. Encuentre los primos de `x`.

> Máximo O(n) → se debe usar punteros.
>
> 
>

```
func ArbolN<Element>::getPrimos(Element: e): Lista<Element>
   Var
       pointer to NodoN<Element>: padre, abuelo, tio, primo
       Lista<Element>: result
   Begin
       result.construir()
       padre <- instance.getPadre(instance.nodoRaiz, e)
       if ¬padre then
           return result
       endif

       abuelo <- instance.getPadre(instance.nodoRaiz, padre->getInfo())
       if ¬abuelo then
           return result
       endif

       tio <- abuelo->getHijoIzq()

       while tio do
           if tio != padre then
               primo <- tio->getHijoIzq()
               while primo do
                   result.insertar(primo->getInfo(), result.getLong() + 1)
                   primo <- primo->getHnoDer()
               endwhile
           endif
           tio <- tio->getHnoDer()
       endwhile

       return result
endfunc

func ArbolN<Element>::getPadre(pointer to NodoN<Element>: ptr, Element: e): pointer
to NodoN<Element>
   Var
       pointer to NodoN<Element>: hijo, padre
   Begin
       if ptr then
           hijo <- ptr->getHijoIzq()
           while hijo do
               if hijo->getInfo() = e then
                   return ptr
               endif
               padre <- instance.getPadre(hijo, e)
               if padre then
                   return padre
               endif
               hijo <- hijo->getHnoDer()
```

```
            endwhile
        endif
        return NULL
    endfunc
```

Recorrido por niveles en zigzag

`func ArbolN<int>::Zigzag(): Lista<int>`

```
func ArbolN<int>::Zigzag(): Lista<int>
   var
       bool : esIzqDer
       Cola<Pointer to NodoN<int>> : colaAux
       Pila<Pointer to NodoN<int>> : pilaAux
       Cola<bool> : colaBool
       Lista<int> : result
       Pointer to NodoN<int> : ptr, hijo
   begin
       colaAux.construir()
       pilaAux.construir()
       colaBool.construir()
       result.construir()

       colaAux.encolar(instance.nodoRaiz)
       colaBool.encolar(true)

       while ¬colaAux.esVacia() do
           ptr <- colaAux.getFrente()
           hijo <- ptr->getHijoIzq()
           esIzqDer <- colaBool.getFrente()

           // Procesar hijos
           while hijo do
               colaAux.encolar(hijo)
               if esIzqDer then
                   pilaAux.apilar(hijo)
               endif
               hijo <- hijo->getHnoDer()
               colaBool.encolar(¬esIzqDer)
           endwhile

           // Inversion del orden de recorrido para hacer zigzag
           if ¬pilaAux.esVacia() ^ ¬esIzqDer then
               result.insertar(pilaAux.getTope()->getInfo(), result.getLongitud() +
1)
               pilaAux.desapilar()
           endif
           if esIzqDer then
               result.insertar(colaAux.getFrente()->getInfo(), result.getLongitud() +
1)
           endif
           colaAux.desencolar()
           colaBool.desencolar()
       endwhile
       return result
endfunc
```

Lowest Common Ancestor

Version clase

```
func ArbolBin<Element>::lca(Element: a, b): Element
```

```
func ArbolBin<Element>::lca(Element: a, b): Element
    Var
        bool: foundA, foundB, lcaSet
        Element: result
    Begin
        foundA <- false
        foundB <- false
        lcaSet <- false
        result <- NULL
        instance.lca(instance.nodoRaiz, a, b, foundA, foundB, lcaSet, result)

        if foundA ^ foundB then
            return result
        endif

        // Si no existen ambos nodos en el arbol, retorna NULL.
        return NULL
endfunc
proc ArbolBin<Element>::lca(pointer to NodoBin<Element>: ptr, Element: a, b, ref bool:
foundA, foundB, lcaSet, ref Element: result)
    Var
        bool: foundAHi, foundAHd, foundBHi, foundBHd
    Begin
        if ptr then
            if ¬ptr->getHijoIzq() ^ ¬ptr->getHijoDer() then
                foundA <- ptr->getInfo() = a
                foundB <- ptr->getInfo() = b
            else
                if ¬foundA v ¬foundB then
                    foundAHi <- false
                    foundAHd <- false
                    foundBHi <- false
                    foundBHd <- false

                    // Procesar Hijos
                    instance.lca(ptr->getHijoIzq(), a, b, foundAHi, foundBHi, lcaSet,
result)
                    if ¬foundAHi v ¬foundBHi then
                        instance.lca(ptr->getHijoDer(), a, b, foundAHd, foundBHd,
lcaSet, result)
                    endif

                    // Procesar Raiz
                    foundA <- foundAHi v foundAHd v ptr->getInfo() = a
                    foundB <- foundBHi v foundBHd v ptr->getInfo() = b

                    if foundA ^ foundB ^ ¬lcaSet then
                        result <- ptr->getInfo()
```

```
                        lcaSet <- true
                    endif
                endif
            endif
        endif
endproc
```

Version simplificada

Corrida en frio

Arbol de ejemplo:

```
          A
        /   \
      B     C
     / \   / \
    D   E F   G
```

Caso: `lca(D, E)`

1. `lcaPtr(A, D, E)`:
   - `A` no es `D` ni `E`.
   - Llama a izquierda y derecha.
2. `lcaPtr(B, D, E)`:
   - `B` no es `D` ni `E`.
   - Llama a izquierda y derecha.
3. `lcaPtr(D, D, E)`:
   - Coincide con `D`.
   - Retorna puntero a `D`.
4. `lcaPtr(E, D, E)`:
   - Coincide con `E`.
   - Retorna puntero a `E`.
5. Regreso a `B`:
   - `izq` retorna `D`, `der` retorna `E`.
   - Como ambos retornan no nulo, `B` es LCA.
   - Retorna puntero a `B`.

6. `lcaPtr(C, D, E)`:
    - No encuentra `D` ni `E` en ese subarbol.
    - Retorna `NULL`.
7. Regreso a `A`:
    - `izq` retorna `B`, `der` retorna `NULL`.
    - Como solo uno es no nulo, propaga `B`.

Resultado final:

- `ptrLca` apunta a `B`.
- `lca(D, E)` retorna `B`.

Caso rapido adicional: `lca(D, G)`

1. Izquierda de `A` retorna `D`.
2. Derecha de `A` retorna `G`.
3. En `A`, ambos lados no nulos → `A` es LCA.

Algoritmo

```
func ArbolBin<Element>::lca(Element: a, b): Element
    Var
        pointer to NodoBin<Element>: ptrLca
    Begin
        ptrLca <- instance.lcaPtr(instance.nodoRaiz, a, b)
        if ptrLca then
            return ptrLca->getInfo()
        endif
        return NULL
endfunc

func ArbolBin<Element>::lcaPtr(pointer to NodoBin<Element>: ptr, Element: a, b):
pointer to NodoBin<Element>
    Var
        pointer to NodoBin<Element>: izq, der
    Begin
        if ¬ptr then
            return NULL
        endif

        if ptr->getInfo() = a v ptr->getInfo() = b then
            return ptr
        endif

        izq <- instance.lcaPtr(ptr->getHijoIzq(), a, b)
        der <- instance.lcaPtr(ptr->getHijoDer(), a, b)

        if izq ^ der then
            return ptr
        endif

        if izq then
            return izq
        endif

        return der
endfunc
```